CSC490A4 - Group 8

Isaac Abell - 1008831329, Thomas(Yehyun) Lee - 1008992217,
Zeling (Zoey) Zhang - 1007714779, Jessica Zhang- 1008494130

Table of Contents

Part One: GRPO and RL Review **1**
Part Two: SFT & Midtraining **2**
2.1 Comparison of pretrain with SFT results 2
2.2 Additional datasets results 2
Part Three: Replicating our RL run with additional analysis **6**
3.1 Comparison plots 6
3.2 Problems review and clustering 7
Part Four: Introducing a more complex rewards system **10**
4.1 Additional RL reward systems 10
4.2 New rewards 10
4.3 Experimental matrix 11
4.4 Quantitative Results 11
4.4.1 Main comparison: original RL vs reward-augmented RL 11
4.4.2 Separate-environment runs 13
4.5 Qualitative examples 15
4.6 Mistakes analysis 15
4.7 Results Summary 18
Works Cited **20**

Part One: GRPO and RL Review

Review the formulation of the rl implementation in nanochat. Write a short paragraph comparing it to standard GRPO [Shao et al., 2024] and why Karpathy may have made the changes he did. Note: before running the training on large machines, replicate the code running locally or on a small GPU so you don't waste credits on false starts.

Karpathy's nanochat RL implementation strips standard GRPO down to its absolute bare minimum, resulting in an architecture that more closely resembles standard REINFORCE with a baseline. He kindly provided descriptions of his RL model's difference from GRPO in the comments of his code. The key architectural changes are as follows:

- **1. Delete trust region, so there is no KL regularization to a reference model:** Standard GRPO utilizes a frozen reference model to calculate a KL divergence penalty, preventing the actor model from drifting into unnatural language to "hack" the reward. Karpathy removes this, which cuts the VRAM requirement in half. This is viable because the training dataset is GSM8K (grade school math); math requires highly specific, verifiable answers, making it difficult for the model to reward-hack.
- **2. Strict On-Policy Training (No PPO Ratio or Clipping):** GRPO typically re-uses generated batches for multiple optimization passes (off-policy), necessitating a probability ratio and clipping function to keep updates stable. Nanochat updates the model's parameters exactly once per batch. Because it is strictly on-policy, the probability ratio is always exactly 1.0, making the calculation and clipping mathematically redundant and a waste of compute.
- **3. Token-Level Normalization:** Instead of sequence-level normalization where the objective is the sum of the log-probabilities, nanochat divides the objective by the total number of valid tokens. In sequence-level REINFORCE, a long answer naturally produces a massive gradient update because more tokens are summed, accidentally teaching the model that generating more tokens yields bigger updates. Token-level normalization fixes this by averaging the update, ensuring a concise correct answer and a long correct answer receive the same magnitude of weight adjustment. However, a potential downside of this approach is that it could disincentivize deep reasoning on harder tasks. By capping the gradient magnitude regardless of the token count, it reduces the learning signal for complex problems that may require longer reasoning.
- **4. Mean Subtraction instead of Z-Score Normalization:** Standard GRPO calculates advantage using a full z-score: $(r-\mu)/\sigma$. Nanochat simplifies this to just $(r-\mu)$. Since GSM8K rewards are binary, a small batch could result in identical rewards for every element in the batch. This would lead to $\sigma=0$, which would cause a division-by-zero error and crash the training loop. Dropping σ from the calculation prevents this. However, it is not clear why he chose this implementation choice over the more traditional method of simply adding a small epsilon to the denominator, as this would make it easier to change the RL training data to more complex datasets with rewards that are not binary.

Part Two: SFT & Midtraining

Note: We are using the nanochat version c7ba25214276d165eeefca7cb2060587975db189 which no longer has midtraining. Thus any future steps do not include midtraining

2.1 Comparison of pretrain with SFT results

Model	Configuration	Final Loss	Val bpb	CORE metric	MFU
Final Nanochat	Nanochat + SwiGLU	2.50	0.75747	0.260	47.4%
Model 2	Nanochat + SwiGLU + SFT	0.83	0.36614	0.322	49.40%

The baseline SFT run produces a substantial improvement over the pretrained model across all reported metrics. The drop in final loss and validation bits-per-byte indicates that the model becomes much better calibrated on the supervised chat-style data distribution, while the improvement in CORE suggests that this gain is not purely token-level but also translates into better downstream chat capability. This is the expected effect of SFT: pretraining gives the model broad linguistic competence, but SFT sharpens that competence into instruction-following behavior and more task-shaped responses. Notably, MFU also increases slightly, suggesting that the fine-tuning stage remains computationally efficient rather than introducing a significant systems penalty. Analytically, this comparison helps establish an important baseline for the rest of the assignment: if later RL improves GSM8K but harms general chat quality, then SFT becomes the key “anchor” phase that gives the model its broad usability before reward optimization begins.

2.2 Additional datasets results

In the current version of the nanochat repository, the mid-training step script has been deprecated in favor of a streamlined Supervised Fine-Tuning (SFT) and Reinforcement Learning (RL) pipeline. To align our methodology with the current codebase, we did not run mid-training during our fine-tuning of the model.

For SFT, we decided to add the following data in order to improve the model.

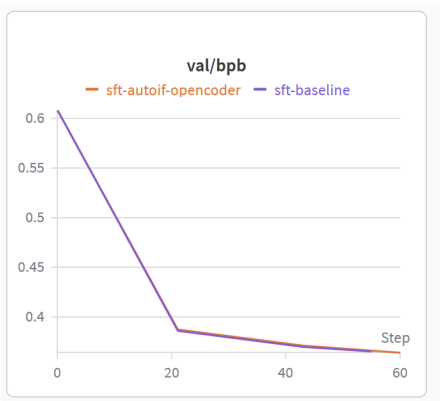
Dataset 1: Coding Improvements - OpenCoder

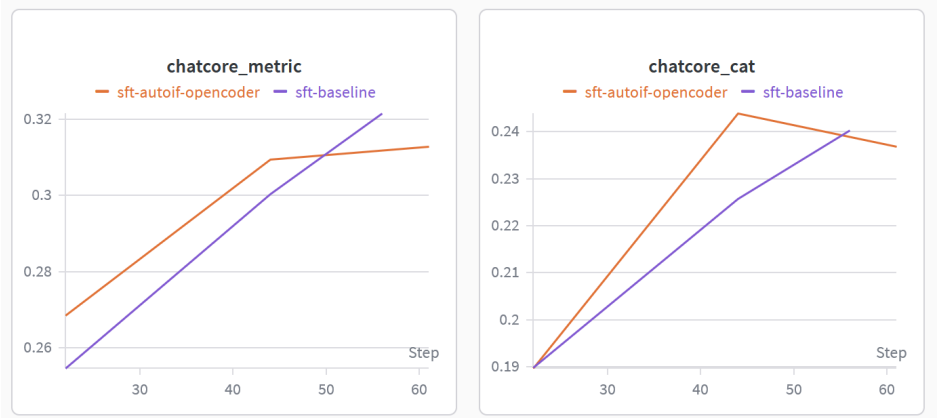
- **Source:** https://huggingface.co/datasets/OpenCoder-LLM/opc-sft-stage2 (OpenCoder-LLM, n.d.)
- **Phase:** SFT
- **Justification:** This dataset is utilized by OpenCoder during their SFT phase. Given the strong performance of OpenCoder models, this data provides high-quality examples for coding tasks. Crucially, the dataset is roughly the same size as our other large training tasks (118k rows). This balanced sizing is intentional; it ensures we do not overwhelm the model with coding data during SFT, which prevents overtraining and avoids **catastrophic forgetting** of previously learned, non-coding tasks.

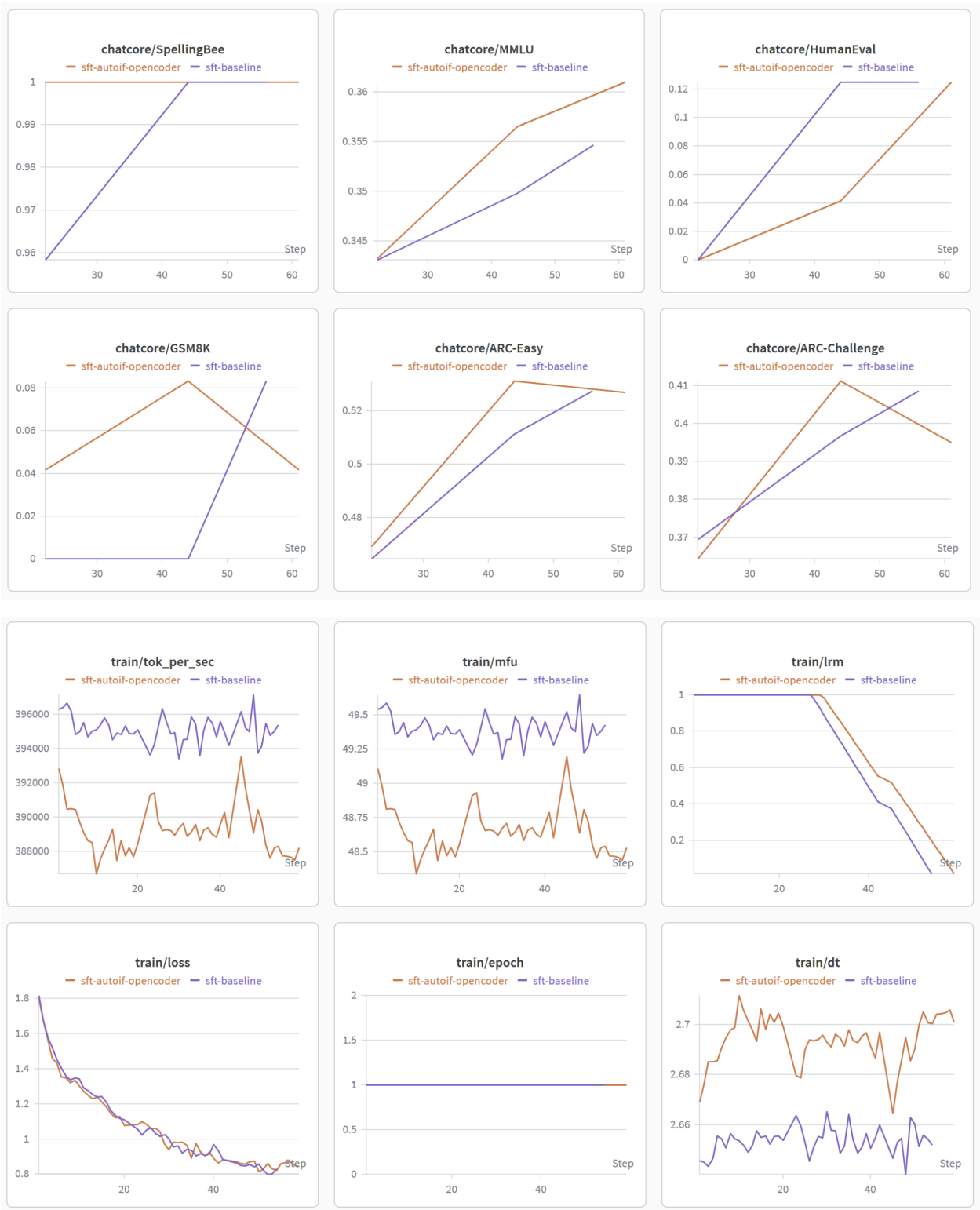
Dataset 2: Instruction Following - AutoIF

- **Source:**https://huggingface.co/datasets/Post-training-Data-Flywheel/AutoIF-instruct-61k-with-funcs (Post-training-Data-Flywheel, n.d.)
- **Phase:** SFT
- **Justification:** Our current SFT data does not adequately enforce strict instruction following. While the model may generate a relevant response to a prompt, it will likely struggle with specific formatting constraints (e.g., "keep this within 100 words"). By introducing this dataset, we train the model to not just answer the prompt, but to strictly adhere to the user's defined boundaries and functional constraints.

Results (WandB)






chatcore/SpellingBee
sft-autoif-opencoder
sft-baseline
Step
chatcore/MMLU
chatcore/HumanEval
chatcore/GSM8K
chatcore/ARC-Easy
chatcore/ARC-Challenge
train/tok_per_sec
train/mfu
train/lrm
train/loss
train/epoch
train/dt

Results (Chat Evaluation and Training resource metrics):

Metric	Model 2 (Baseline SFT)	Model 3 (Baseline SFT + AutoIF, OpenCoder)
Architecture / Data	SFT	SFT + AutoIF, OpenCoder
Final Loss	0.832	0.844
Val bpb	0.3661	0.3646
CORE metric	0.3099	0.3135
HumanEval	9.76%	12.80%
MMLU	35.42%	36.08%
ARC-Easy	52.78%	52.53%
ARC-Challenge	40.44%	39.42%
GSM8K	5.84%	5.76%
SpellingBee	98.83%	98.83%
MFU	49.43%	48.53%
Training Time (minutes)	21.7	24.42
Compute Cost	$5.71	$6.42

Observations & Evaluation: As expected, the addition of the new datasets increased the total training time from 21.7 minutes to 24.42 minutes on our 4xH100 setup due to the larger volume of data ingested. We observed that while the losses decreased at a similar rate between the two runs, MFU was slightly worse, and performance occasionally fluctuated during intermediate evaluations, potentially indicating brief periods of overtraining or distribution shifts. Ultimately, the final evaluation metrics demonstrate that the datasets successfully achieved their targeted goal:

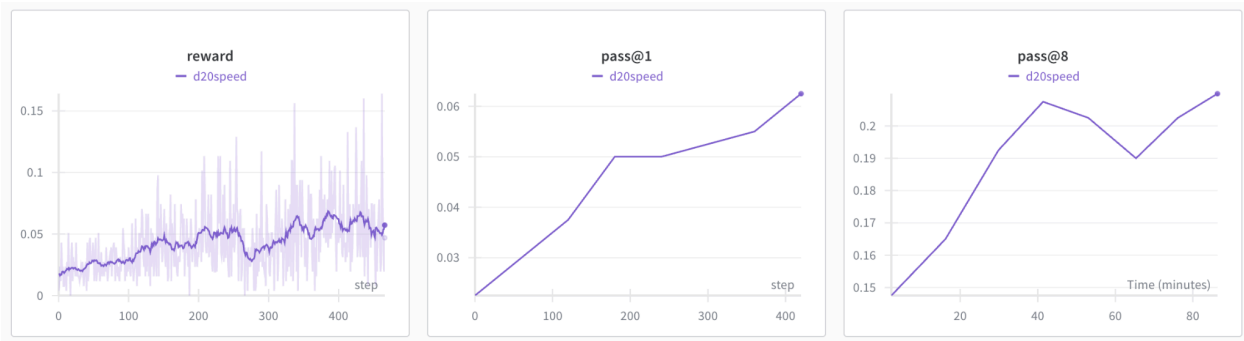
- **Coding Improvements:** The model's coding ability, measured by the HumanEval benchmark, saw a significant improvement, jumping from 9.76% to 12.80%.
- **General Reasoning:** We saw a mild improvement in MMLU accuracy (35.42% → 36.08%), suggesting the strict logic required by the coding datasets had a positive halo effect on general reasoning.
- **Trade-offs:** We observed a minor alignment tax on unrelated tasks, with ARC-Easy and ARC-Challenge seeing slight regressions (dropping by ~0.25% and ~1.02% respectively), while math reasoning (GSM8K) remained effectively unchanged.

Overall, the aggregate ChatCORE metric improved from 0.3099 to 0.3135. The results confirm that inserting high-quality, targeted data like OpenCoder effectively enhances specific capabilities like code generation, validating our data curation strategy.

Part Three: Replicating our RL run with additional analysis

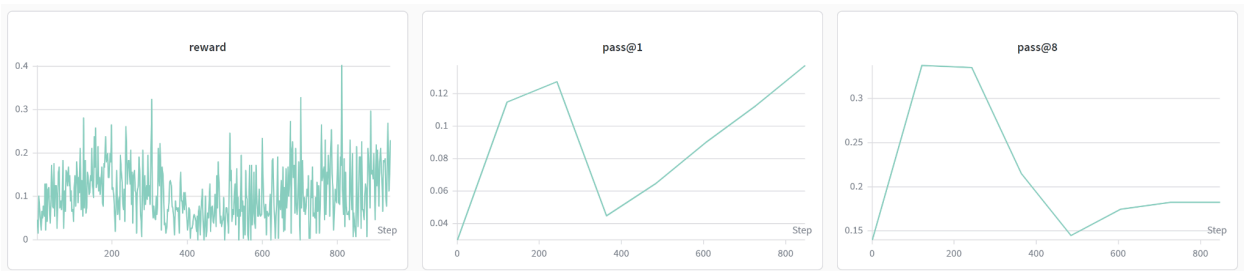
3.1 Comparison plots

Original run(Purple) - GSM8K accuracy: 7.58%



Our run (Nanaochat + Swiglu + SFT + RL) - GSM8K accuracy: 12.43%

We chose the checkpoint without the two additional datasets since GSM8K is math problems only, not related to coding or instruction following (the chatbot has to come up with the solution steps)



Difference analysis:

Difference 1: reward trend

- We see in the original run there is a slight dip in reward around step 250. We also see a similar dip around the same area possible due to a lSargely similar model structure as the only difference we had was using SwiGLU instead of RELU. However **the original run's reward drop has a smaller negative impact on the performances.** In pass@1 there seems to be no effect (the accuracy continues to rise at ~250 steps), and in pass@8 there was some decrease (~0.01-0.02, assuming the x axes in pass@1 and pass@8 are aligned). **Our run on the other hand has a much stronger effect on our performance,** and our pass@k performance pretty much closely follows the reward trend. Our reward trend has a drastic increase in the first 200 steps, which is reflected in the slope of both pass@1 and pass@8. Similarly at around 300 steps we see a drastic drop of reward, which is reflected in both pass@1 and pass@8.

Difference 2: pass@k trend

- We see that the overall trend of pass@1 in the original run is non-negative, and the pass@8 shows a small dip but soon recovers. However for our run, pass@1 suffered from a significant performance decrease, and took ~100 steps to recover to an upward trend. It was not able to exceed the former peak performance until ~500 steps later. pass@8 on the other hand suffered from a similar drop of performance, and was not able to recover to performance better than the previous peak.
- We also see the slope of our run is much steeper than the original run.

Difference 3: pass accuracies

- Our accuracy is significantly higher than the original(+~0.07 for pass@1 and +~0.13 for pass@8), with ours at step 200 being ~.12 for pass@1 and ~0.33 for pass@8, while the original at step 200 is ~0.05 for pass@1 and ~0.2 for pass@8.

Overall we infer that these are reflections of the SwiGLU mechanism, demonstrating early high performance and showing step-efficiency (steep gains), however the downside is that it is more sensitive to reward fluctuations.

3.2 Problems review and clustering

Analysis 1: Bug Catching

Some results had formatting error (missing #### for answer, failed to convert words to numbers etc.)

Example:

```
"question": "Adrianne is collecting different kinds of beads for making
bracelets. Her mother gave her 20 metallic beads. Her sister gave her ten
more beads than her mother, and her friend gave her twice as many as her
mother gave.  How many beads did Adrianne have altogether? Adrianne
received 20 +10 =  30 beads from her sister.\nAnd, she received 2 x 20 =
40 beads from her friend.\nTherefore, Adrianne had 20 + 30 + 40 =  90
beads altogether.\n#### 90",
"completion": "20 + ten =[CALC] 20+ ten"
```

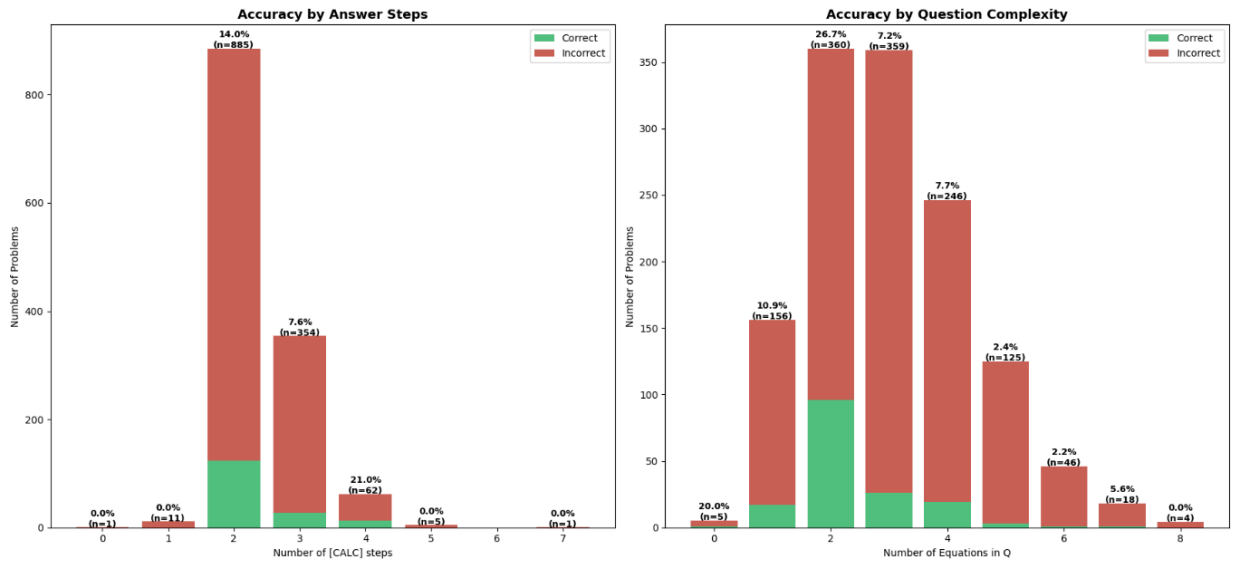
What likely happened: the incorrect conversion from ten to 10, causing an issue in [calc] function call, preventing the generation to go on and thus the response halts and doesn't generate \n#### for answer, which is what the eval script looks for. Though the train of thought could be correct, this error led to failed answer generation. In part 4, we can add reward on having the right format since randomly sampled qualitative analysis of function calls seems to yield correct results.

Analysis 2: Depth analysis

Here we wanted to see if the model is able to understand the question and apply the right amount of steps, in other words, if the path to solution is optimal. Thus we plotted the depth of the generated answer (calculated by the amount of [CALC] function calls, left graph) and the depth of the sample answer (calculated by the number of equations in questions, right graph).

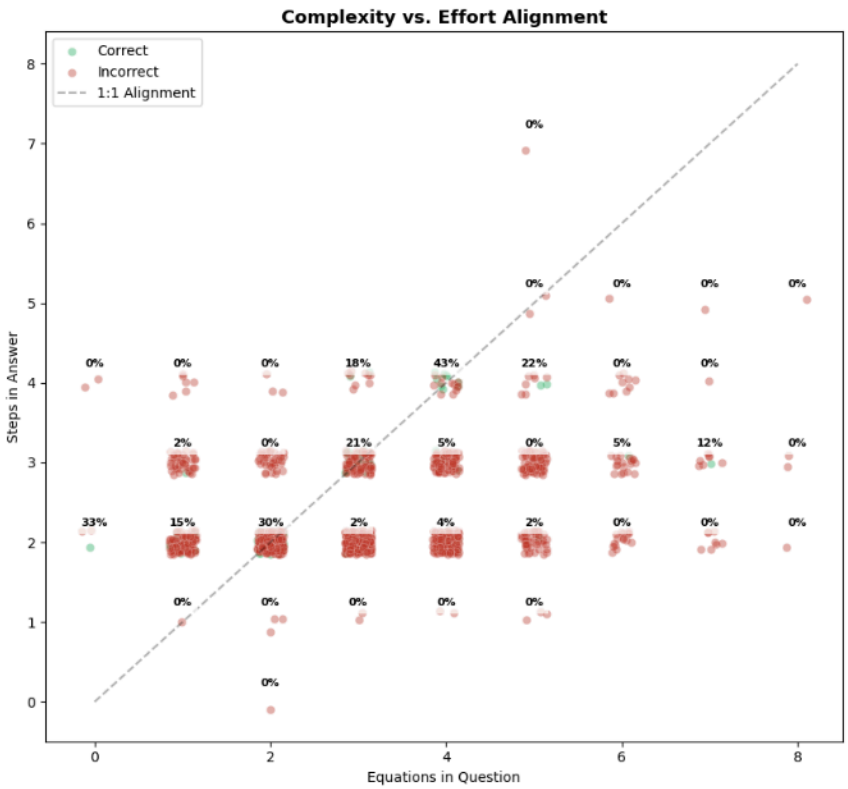
We observe:

1. The model doesn't really understand how to convert the problem into steps and prefers lower depth. From the left graph, the model highly prefers calling calc 2 or 3 times, while the correct answer has a more even distribution with simple questions and complex questions.
   Evidence in graph:



	Sample answer	Model answer
2→3 ratio	~1 to 1 (360 to 359)	~1 to 0.4 (885 to 354)
3→4 ratio	~1 to 0.69 (359 to 246)	~1 to 0.18 (354 to 62)
Simple answer counts (d<2)	161	12
Complex answer counts (d>4)	209	6

2. From the below graph we defined the optimal alignment of question to answer depth (1:1) and we see that the clusters that lies on or close to that line has a high accuracy, verifying our hypothesis that the 1:1 steps could lead to improved performance and is a good indicator of correct understanding and conversion. Thus in part 4, we could improve our reward system by rewarding intermediate steps or a similar amount of depth, similar to how chain of thought(CoT) improved LLM performance. (Wei et al., 2022)


Complexity vs. Effort Alignment
Correct
Incorrect
1:1 Alignment
Steps in Answer
Equations in Question
0%
0%
0%
0%
0%
0%
0%
0%
0%
18%
43%
22%
0%
0%
2%
0%
21%
5%
0%
5%
12%
0%
33%
15%
30%
2%
4%
2%
0%
0%
0%
0%
0%
0%
0%
0

Part Four: Introducing a more complex rewards system

4.1 Additional RL reward systems

Part 3 showed that the original GSM8K RL setup has two distinct failure modes. First, some answers fail for **structural reasons** rather than reasoning quality: the model omits the required #### number ending, produces malformed outputs, or leaves [CALC] traces unresolved. Second, many incorrect solutions fail for **reasoning-depth reasons**: the model stops after only 2–3 operations, even when the reference solution requires a longer chain of transformations. These observations suggest that the original binary correctness reward under-specifies desirable behaviour. It only distinguishes final success from final failure, and therefore provides no gradient signal for intermediate qualities such as answer formatting, complete tool usage, or adequate reasoning depth.

To test whether richer reward shaping can address these weaknesses, we introduced two additional rewards and reran RL from the same SFT checkpoint. We evaluate them in two ways. First, we add each reward to the original correctness-based RL objective to test whether it improves GSM8K under the standard setup. Second, we run each reward in its own environment to isolate its effect and study whether the reward can independently shape model behaviour. This two-level design lets us separate **additive improvement** from **standalone behavioural influence**, which is more informative than reporting only a single modified reward run.

4.2 New rewards

Reward	Goal	Key knobs	Hypothesis
Reward A (format)	Reward A aims to encourage structurally valid answers with complete formatting and resolved [CALC] usage rather than mathematical correctness. It gives a **positive bonus** when the response: ● Ends with a parseable #### number, ● Remains under a capped length budget (220 tokens), ● and Resolves all [CALC] calls cleanly. It applies **penalties** for missing the final answer marker,	bonus=+0.2, penalty=-0.2, length_pen=-0.05, unresolved=-0.1, max_len=220	Reward A should sharply reduce formatting failures and incomplete generations, but may not improve mathematical accuracy by much because it does not directly evaluate whether the reasoning chain is correct.

	overlong responses, and unresolved calculator traces.		
Reward B (depth)	Reward B targets under-reasoning by encouraging responses whose reasoning depth more closely matches the teacher's solution. It compares the model's reasoning depth to the reference solution depth and gives: • a **large bonus** for close alignment, • a **smaller bonus** for partial alignment, • **a penalty** for responses that are too shallow.	depth_bonus =+0.2, depth_mild_bonus=+0.1, depth_penalty=-0.15	Reward B should improve GSM8K more than Reward A, as GSM8K errors in Part 3 were often caused by prematurely short reasoning traces. However, depth shaping may also introduce a failure mode of its own: the model may generate unnecessary extra steps or "padding" to satisfy the reward proxy without improving the underlying math.

4.3 Experimental matrix

All runs were trained on Modal (**stage_eval**) using the same 4×H100 setup to keep hardware constant across comparisons. We evaluate four Part 4 variants in addition to the Part 3 baseline:

1. **Baseline RL (Part 3)**: original correctness reward only
2. **Baseline + Reward A**: original RL objective plus format reward
3. **Baseline + Reward B**: original RL objective plus depth reward
4. **Reward A environment**: Reward A isolated in its own environment
5. **Reward B environment**: Reward B isolated in its own environment

4.4 Quantitative Results

4.4.1 Main comparison: original RL vs reward-augmented RL

GSM8K eval output (Baseline + Reward A):

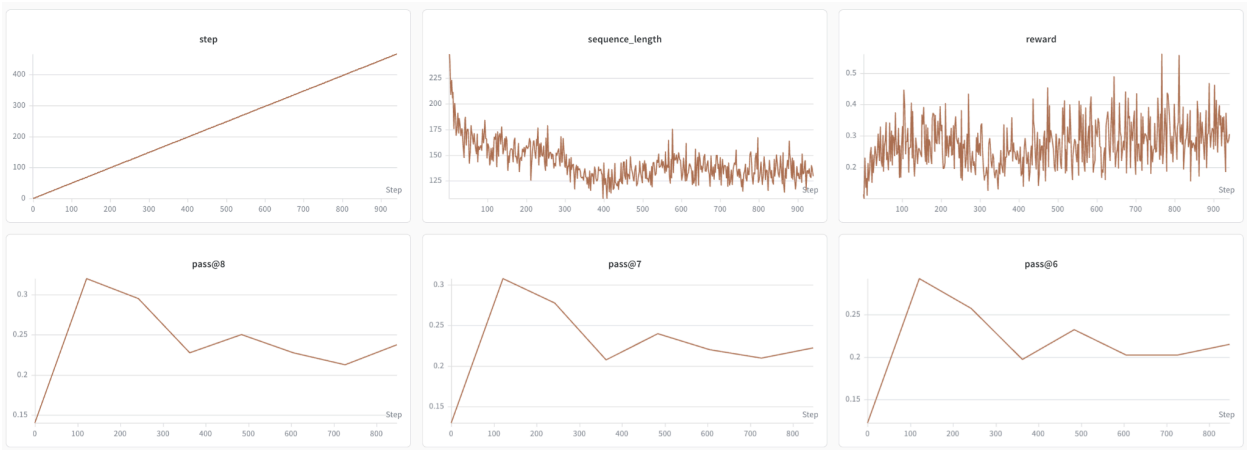
```
Rank 0 | 152/1276 (11.91%)⁚ Running (1/1 containers act
Rank 0 | 154/1319 (11.68%)
==================================================
Final: 154/1319 (11.68%)
GSM8K accuracy: 11.68%
```

GSM8K eval output (Baseline + Reward B):

```
Rank 0 | 157/1028 (15.27%): Running (1/1 containers activ
Rank 0 | 161/1077 (14.95%): Running (1/1 containers activ
Rank 0 | 170/1130 (15.04%): Running (1/1 containers activ
Rank 0 | 174/1182 (14.72%): Running (1/1 containers activ
Rank 0 | 182/1233 (14.76%): Running (1/1 containers activ
Rank 0 | 187/1283 (14.58%): Running (1/1 containers activ
Rank 0 | 193/1319 (14.63%)
==================================================
Final: 193/1319 (14.63%)
GSM8K accuracy: 14.63%
```

Baseline + Reward A:

Baseline + Reward B:



Run	Reward trend notes	Pass@8 peak (train)	GSM8K eval (%)
Baseline RL (Part 3)	Smooth climb with mild dip near step 400	~0.33	12.43

Baseline + Reward A	Reward spikes early, stays around; sequence length shrinks from 225 to <125 tokens	~0.27	11.68
Baseline + Reward B	Reward trend is similar to Baseline + Reward A, but with slight volatility; depth shaping keeps sequences near 130 tokens	~0.30	14.63

The main result is that the two added rewards have very different effects. Reward A successfully changes output structure, but does not improve task accuracy; in fact, GSM8K drops slightly from **12.43% to 11.68%**. This suggests that on GSM8K, fixing surface-form errors is not sufficient to improve final mathematical performance. By contrast, Reward B improves accuracy to **14.63%**, the best result among all Part 4 additive-reward runs. This supports the hypothesis that the larger bottleneck is not only formatting, but inadequate reasoning depth.

An important analytical takeaway is that **reward shaping is not uniformly beneficial**. Reward A optimizes behavior that is easy to measure and visibly improves output cleanliness, but because it is largely orthogonal to mathematical correctness, it yields little transfer to GSM8K accuracy. Reward B, however, targets a proxy more tightly coupled to success on multi-step arithmetic problems, and therefore produces a meaningful gain.

4.4.2 Separate-environment runs

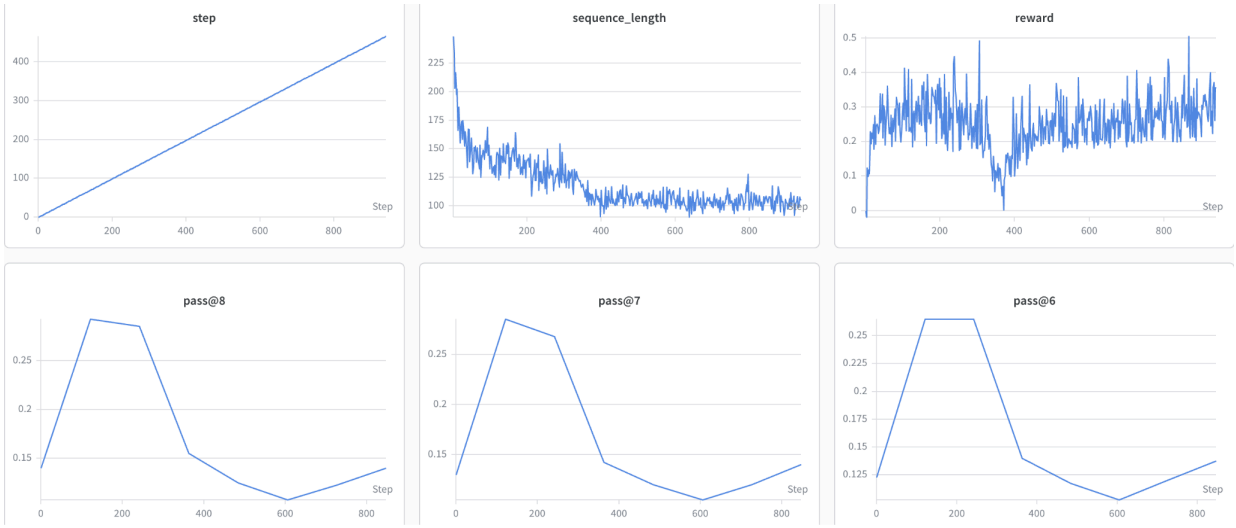
GSM8K eval output (Reward A environment):

```
Rank 0 | 116/1293 (8.97%): Running (1/1 containers active)...
Rank 0 | 118/1319 (8.95%)
==================================================
Final: 118/1319 (8.95%)
GSM8K accuracy: 8.95%
```

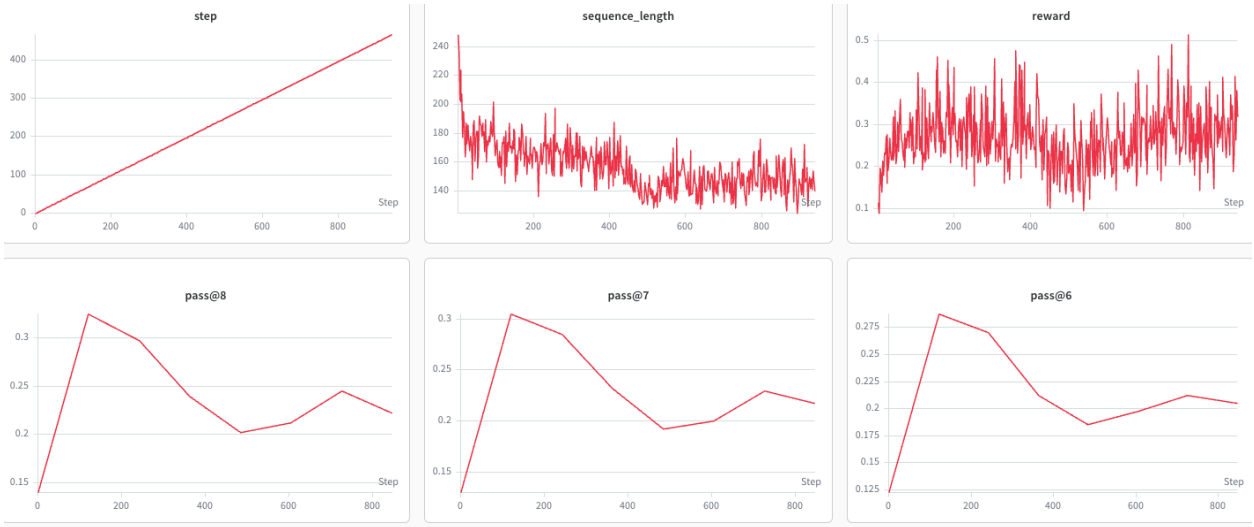
GSM8K eval output (Reward B environment):

```
Rank 0 | 174/1276 (13.64%): Running (1/1 containers active).
Rank 0 | 177/1318 (13.43%)Rank 0 | 177/1319 (13.42%)
==================================================
Final: 177/1319 (13.42%)
GSM8K accuracy: 13.42%
```

Reward A environment:



Reward B environment:



Run	Reward trend notes	GSM8K Eval
Reward A Env	Sequence length rapidly compresses; reward stabilizes; pass@8 peaks early then declines	8.95
Reward B Env	Longer sequences persist; reward noisier; pass@8 declines after early peak but remains stronger than Reward A Env	13.42

The separate-environment results strengthen this interpretation. Reward A in isolation performs substantially worse than the baseline and worse than its additive counterpart, indicating that

formatting-oriented shaping does not form a sufficient standalone objective for GSM8K. Reward B in isolation reaches **13.42%**, better than the original nanochat RL run (**7.58%**) but still below **Baseline + Reward B (14.63%)**. This suggests that depth shaping is useful, but works best when anchored by the original correctness reward rather than replacing it as the dominant behavioral signal.

This is a useful systems-level result: in small-model RL, **a good auxiliary reward is often most effective as a shaping term rather than a replacement objective**.

4.5 Qualitative examples

Example 1: Reward A fixes structure, not reasoning

Prompt. Adrianne has 20 beads from her mom, 10 more from her sister, and twice as many from her friend. How many beads does she have in total?

Reward A output.

```
She has 20+10=30 beads. She has 30*2=60 beads. #### 60
```

This example illustrates the intended effect of Reward A: the output is short, complete, and ends with a valid final answer marker. The formatting objective is clearly working. However, the reward provides no guarantee that the reasoning is mathematically complete across harder examples; it primarily cleans up the surface form of the output.

Example 2: Reward B increases depth, but can still fail late

Prompt. A car drives 45 miles in 3 hours, then 70 miles in 2 hours. What is the average speed?

Reward B output.

```
45×3=135 miles. 70×2=140 miles. 135+140=275. #### 275
```

This example shows the strengths and limitations of Reward B. The response now exhibits a longer reasoning chain and multiple intermediate computations, consistent with the reward's objective. However, the model still fails on the final conceptual step: average speed requires dividing total distance by total time. This is a stronger failure than a missing ####, but still a failure. It indicates that Reward B improves decomposition more than it improves full solution correctness.

4.6 Mistakes analysis

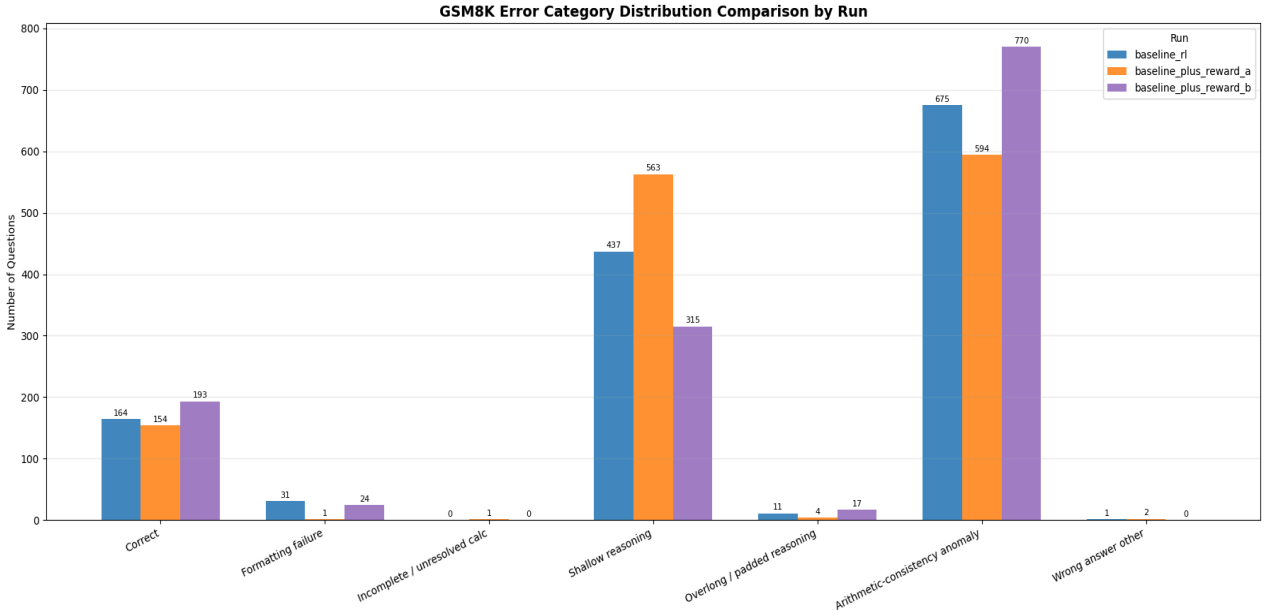
To move beyond raw accuracy, we compare how the dominant error types shift under each reward. Because time constraints prevented full human annotation, we use an **automated error taxonomy** based on deterministic heuristics over the final answer marker, calculator usage,

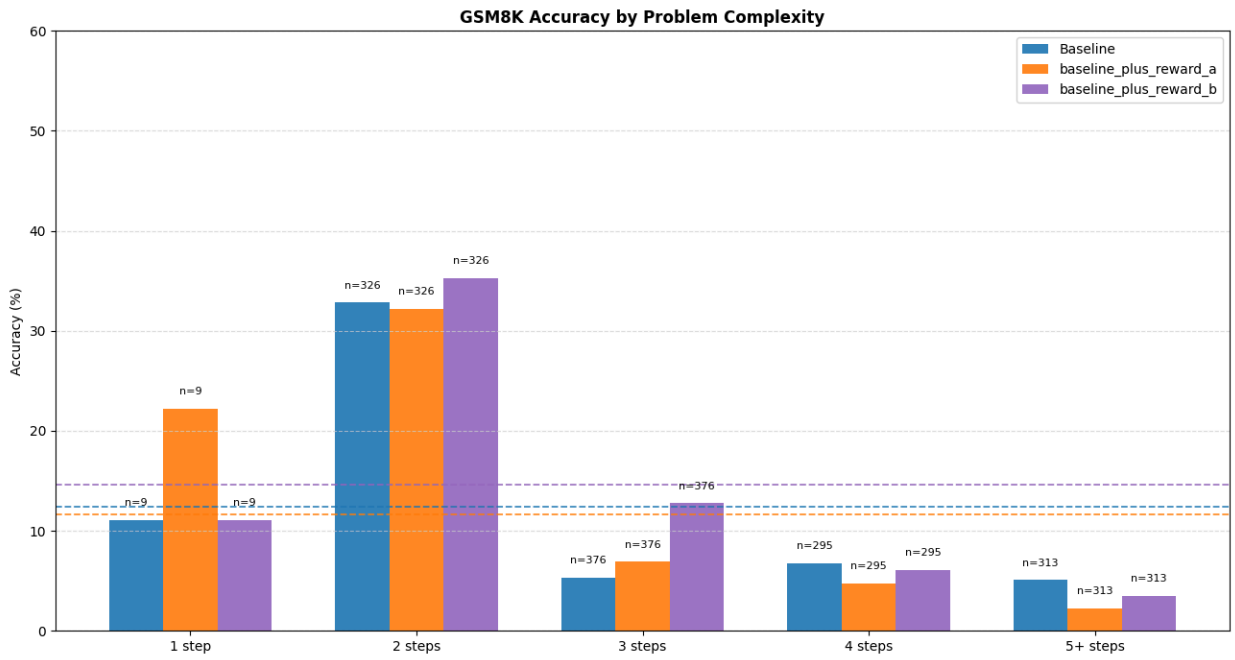
response length, and reasoning depth relative to the reference solution. Each completion is assigned one dominant category.

<u>Automated error taxonomy</u>

- **Formatting failure**: missing or malformed `#### number`
- **Incomplete/unresolved calc**: malformed or unfinished `[CALC]` usage
- **Shallow reasoning**: reasoning depth substantially below the reference solution
- **Overlong / padded reasoning**: reasoning depth substantially above the reference
- **Arithmetic-consistency anomaly**: internal arithmetic trace inconsistent with reported intermediate results
- **Wrong answer other**: incorrect outputs not captured by the above rules

This automated taxonomy is approximate rather than semantically perfect, but it is sufficient to reveal the main behavioural shifts caused by the reward change.





Across the three runs, the failure taxonomy and the complexity breakdown tell a consistent story about what each reward is actually changing. The baseline RL model produces 164 correct GSM8K answers, and its dominant failures are shallow reasoning (437 questions) and arithmetic-consistency anomalies (675 questions), with a smaller but still nontrivial formatting-failure tail (31 questions). The complexity chart helps localize where those failures show up behaviorally. Baseline accuracy is strongest on 2-step problems (32.8%) but drops off quickly once longer reasoning is required: 5.3% on 3-step, 6.8% on 4-step, and 5.1% on 5+ step problems. In other words, even before looking at the taxonomy, the baseline already appears bottlenecked by insufficient reasoning depth outside the easiest multi-step bucket.

Reward A improves the output surface but not the underlying reasoning competence. It cuts formatting failures from 31 to 1, which strongly confirms that the format reward is doing what it was designed to do. That improvement shows up most clearly on the simplest problems: 1-step accuracy rises sharply from 11.1% to 22.2%. But the gains do not carry over to the harder buckets. Reward A is slightly worse than baseline on 2-step problems (32.2% vs. 32.8%), only modestly better on 3-step problems (7.2% vs. 5.3%), and worse again on 4-step and 5+ step questions. This lines up with the failure counts: correct answers actually fall from 164 to 154, while shallow-reasoning failures rise substantially from 437 to 563. The most likely mechanism is that Reward A's 220-token cap suppresses continuation on problems that require longer chains of thought. So, although it cleans up formatting noise and modestly reduces arithmetic anomalies from 675 to 594, it does so partly by preventing the model from attempting enough reasoning in the first place. In effect, Reward A trades visible surface errors for deeper truncation errors. It makes answers look cleaner, but on GSM8K that cleanliness is not the main bottleneck.

Reward B produces a qualitatively different shift and is much better aligned with the task. It yields 193 correct answers, the best overall result, and reduces shallow-reasoning failures from 437 to 315. The complexity chart makes this especially convincing: Reward B is best on 2-step problems (35.3% vs. 32.8% baseline) and shows the clearest gain on 3-step problems (13.0% vs. 5.3% baseline), the first bucket where reasoning depth becomes much more demanding. It is not uniformly better in every category — it is slightly below baseline on 4-step and 5+ step questions — but its overall improvement comes from materially strengthening performance in the medium-complexity region where a large portion of solvable GSM8K questions live. That behavioural shift matches the taxonomy exactly: the model is less likely to fail by stopping too early, and more likely to at least attempt the necessary reasoning structure. The tradeoff is that arithmetic-consistency anomalies rise from 675 to 770, the highest of the three runs, and overlong or padded reasoning increases from 11 to 17. But that tradeoff is coherent. Once the model is encouraged to continue reasoning further, it naturally creates more opportunities to make arithmetic slips deeper in the chain.

Together, the two analyses show that reward A mostly improves syntactic validity and helps with very shallow questions, but it worsens the model's main bottleneck by increasing shallow-reasoning failures on the broader task. Reward B, by contrast, directly attacks that bottleneck: it improves accuracy precisely in the multi-step regime where the baseline begins to collapse, and the resulting failures shift from "did not reason far enough" to "reasoned further but made arithmetic mistakes." For GSM8K, that is a much healthier failure mode. Arithmetic inconsistencies inside a mostly correct reasoning trace are a narrower and more tractable weakness than wholesale premature termination.

4.7 Results Summary

Variant	GSM8K (%)	Training dynamics	Main impact	Main limitation	Dominant error (Count)
Baseline RL (Part 3)	12.43	Smooth climb with mild dip mid-training	Strong reference baseline after checkpoint fix	Still suffers from structural errors and shallow chains	Arithmetic anomaly (675)
Baseline + Reward A	11.68	Sequence length compresses sharply; reward stabilizes early	Greatly improves answer formatting and completion behavior	No accuracy lift; reward is weakly coupled to math correctness	Shallow reasoning (563)
Baseline + Reward B	14.63	Slightly noisier reward; longer reasoning persists	Best overall GSM8K; improves	Some remaining arithmetic slips and occasional padded steps	Arithmetic anomaly (770)

			reasoning depth		
Reward A environment	8.95	Strong brevity pressure; early peak then decline	Shows format reward can independently shape structure	Too weak as a standalone objective	-
Reward B environment	13.42	Longer traces and noisier reward	Depth reward remains useful in isolation	Still worse than using depth as an auxiliary shaping reward	-

Final takeaway

The main lesson from Part 4 is that an auxiliary reward only helps when it is tightly aligned with the task's actual failure bottleneck. In our GSM8K setting, the baseline model does not primarily fail due to formatting noise; it fails because it often stops too early or does not carry reasoning far enough through multi-step problems. Reward A is easy to optimize and clearly improves output hygiene, but that improvement is largely superficial for this benchmark: it removes formatting failures without improving overall task accuracy, and slightly worsens it. Reward B is more task-aligned. By encouraging longer and more complete reasoning traces, it reduces shallow-reasoning failures and improves performance, particularly on multi-step problems, where the baseline begins to break down. Its cost is an increase in arithmetic-consistency errors, but it remains a more productive failure regime than premature truncation. Overall, the results suggest that for small-model RL on reasoning tasks, structural rewards can be useful as shaping terms only when they reinforce the benchmark's true cognitive demand, here, sustained reasoning depth, rather than merely polishing surface form.

Works Cited

OpenCoder-LLM. (n.d.). *opc-sft-stage2* [Data set]. Hugging Face. https://huggingface.co/datasets/OpenCoder-LLM/opc-sft-stage2

Post-training-Data-Flywheel. (n.d.). *AutoIF-instruct-61k-with-funcs* [Data set]. Hugging Face. https://huggingface.co/datasets/Post-training-Data-Flywheel/AutoIF-instruct-61k-with-funcs

Wei, J., Wang, X., Schuurmans, D., Bosma, M., Fei-Fei, L., Chi, E. H., Chen, F., & Zhou, D. (2022). Chain-of-thought prompting elicits reasoning in large language models. *arXiv preprint arXiv:2201.11903*. https://doi.org/10.48550/arXiv.2201.11903